ID	Module	Function	Description	Input / Condition	Expected / Result
1	AnimalCategory	Constructor (int, String, String)	Creates an animal category with id, type, and description.	Valid id, type, description	Returns a fully initialized AnimalCategory object.
2	AnimalCategory	getId()	Returns the category ID.	Called on valid object	Returns int id.
3	AnimalCategory	getType()	Returns the category type.	Called on valid object	Returns String type.
4	AnimalCategory	getDescription()	Returns the category description.	Called on valid object	Returns String description.
5	AnimalCategory	setType(String)	Updates the type field.	New type value	Type field updated.
6	AnimalCategory	setDescription(String)	Updates the description field.	New description value	Description updated.
7	AnimalCategory	toString()	Provides string representation.	Called on object	Returns "Category: " + type + " - " + description.
8	Customer	Default constructor	Creates an empty customer.	No parameters	Empty Customer with default fields.
9	Customer	Constructor (int,String,String,String)	Creates a customer with id, name, address, phone.	id, name, address, phone	Returns initialized Customer.
10	Customer	toString()	Basic string representation.	Called on object	"name (phone)".

11	Employee	Default constructor	Creates an empty employee.	No parameters	Empty Employee.
12	Employee	Constructor (int,String,String)	Creates employee with id, name, role.	id, name, role	Initialized Employee.
13	Employee	toString()	String representation for employee.	Called on object	`"Empleado: name
14	Inventory	Field products	Internal storage of product list.	Inventory instantiated	products initialized as empty list.
15	Inventory	Field gson	JSON serializer/deserializer.	Inventory created	gson available for JSON operations.
16	Inventory	addProduct(Product)	Adds product if non-null.	product != null	Product added to list.
17	Inventory	addProduct(Product) (null)	Tries to add null.	product == null	No action taken.
18	Inventory	getNextProductId() (deprecated)	Computes next numeric ID by extracting digits.	Products exist or not	Returns max+1; ignores non-numeric patterns.
19	Inventory	getProducts()	Returns the products list.	Called	Returns reference to internal list.
20	Inventory	showInventory()	Prints inventory overview to console.	Product list empty or not	Prints "No products..." or prints each product.
21	Inventory	safe(String)	Protects against null strings.	s null or not	Returns "" if null; else returns same string.

22	Inventory	updateStock(String,int)	Updates stock for exact matching product ID.	ID & quantityChange	If ID not found → message. If new stock <0 → warning. Else update + warnings for 0 or low stock.
23	Inventory	updateStock (not found)	ID doesn't match any product.	Invalid ID	Prints "Product not found: ID".
24	Inventory	updateStock (negative stock)	quantityChange leads to stock < 0.	e.g. subtracting too much	Prints "Not enough stock."
25	Inventory	updateStock (stock = 0)	Stock reaches zero.	newStock == 0	Prints "Product out of stock."
26	Inventory	updateStock (low stock)	Stock < 4 and >0.	newStock < 4	Prints "Low stock warning."
27	Inventory	findProductByName(String)	Case-insensitive exact name match.	name null or string	Returns product or null.
28	Inventory	findProductsByName(String)	Partial search (contains, case-insensitive).	partial null or string	Returns empty list or list of matches.
29	Inventory	sellProductInteractive(Scanner)	Interactive selling flow.	Scanner input: name + quantity	Validates name, stock, quantity; updates stock.
30	Inventory	sellProductInteractive (partial match)	Name partially matches product.	substring of name	First product that matches is selected.
31	Inventory	sellProductInteractive (non-number quantity)	Quantity input not numeric.	e.g. "abc"	Prints "Invalid quantity."
32	Inventory	saveToJson(String)	Saves product list to a JSON file.	valid path	Writes file or prints error if fails.

33	Inventory	saveToJson (error)	Save operation fails.	IO exception	Prints “Error saving products.json: …”.
34	Inventory	loadFromJson(String)	Loads product list from JSON.	File exists and valid	Replaces products with loaded list.
35	Inventory	loadFromJson (missing file)	JSON file not found.	Path invalid	No action taken.
36	Inventory	loadFromJson (bad JSON)	Corrupted or wrong format JSON.	Invalid file	Prints error loading message.
37	Inventory	generateReport()	Prints detailed inventory report.	products empty or not	Displays products + total count.
38	Inventory	modifyInventoryByCategory(Scanner,String)	Filter by category and modify price/stock.	scanner inputs for category → ID → option	Updates selected product and saves JSON.
39	Inventory	modifyInventoryByCategory (empty category)	Category input is empty.	user input ""	Prints “Empty category. Cancelling.”
40	Inventory	modifyInventoryByCategory (no matches)	No products in that category.	category doesn’t exist	Prints “No products found in that category.”
41	Inventory	modifyInventoryByCategory (empty ID)	User doesn’t enter an ID.	ID=""	Cancels operation.
42	Inventory	modifyInventoryByCategory (invalid option)	Option not 1 or 2.	user input “3” or other	Prints “Invalid option.”
43	Invoice	Default constructor	Creates an empty invoice.	No parameters	Empty invoice object.
44	Invoice	Constructor (int,Date,double,	Creates full invoice.	All fields provided	Invoice initialized.

		Customer,Order, Employee)			
45	Invoice	toString()	Summary representation of invoice.	Called	`"Invoice #id
46	Order	Default constructor	Creates order with empty detail list.	No parameters	details = new ArrayList<>().
47	Order	Constructor (int,Customer,Employee,Date)	Creates an order with info and empty details.	id, customer, employee, date	New Order object.
48	Order	addDetail(OrderDetail)	Adds one item/detail to order.	valid detail	Detail added to list.
49	Order	calculateTotal()	Computes total by summing all subtotals.	details list 0..n items	Returns total (0 if empty).
50	Order	toString()	Summary of order including total.	Called	`"Order #id
51	AnimalCategory	setDescription()	Updates description	Valid object + new description	Description updated
52	AnimalCategory	equals()	String representation	Different fields	Returns true
53	AnimalCategory	equals()	Change customer name	Called on instance	Returns true
54	AnimalCategory	toString()	Change email	Called on instance	Returns formatted string
55	Customer	setName()	Invalid email	Valid name	Name updated
56	Customer	setEmail()	Change email	Valid email	Email updated
57	Customer	setEmail()	Invalid email	Wrong format	Throws IllegalArgumentException

58	Customer	deactivate()	Soft delete	Valid customer	Status becomes inactive
59	Customer	activate()	Reactivate	Inactive customer	Status becomes active
60	Customer	equals()	Compare customers	Same id	Returns true
61	Customer	equals()	Compare customers	Different id	Returns false
62	Employee	setSalary()	Update salary	Positive value	Salary updated
63	Employee	setSalary()	Update salary	Negative value	Throws exception
64	Employee	promote()	Increases position rank	Valid employee	Rank increases
65	Employee	calculateAnnual Salary()	Computes salary	Monthly salary given	Annual = monthly × 12
66	Employee	equals()	Compare two employees	Same id	Returns true
67	Inventory	setCategory()	Compare two employees	Valid category	Category updated
68	Inventory	setCategory()	Null category	category = null	Throws exception
69	Inventory	reduceStock()	Subtract amount	Enough stock	Stock decreases
70	Inventory	reduceStock()	Subtract amount	Not enough stock	Throws exception
71	Inventory	increaseStock()	Add to stock	Valid amount	Stock increases
72	Inventory	isLowStock()	Check threshold	qty < threshold	Returns true
73	Inventory	isLowStock()	Check threshold	qty ≥ threshold	Returns false

74	Inventory	equals()	Compare inventory	Same item & category	Returns true
75	Invoice	setCustomer()	Assign new customer	Valid customer	Updates customer
76	Invoice	addItem()	Add order item	Valid item	Item list increases
77	Invoice	addItem()	Add item	Duplicate item	Quantity increments
78	Invoice	removeItem()	Remove item	Item exists	Item removed
79	Invoice	removeItem()	Remove nonexistent item	Not found	No change
80	Invoice	calculateTax()	Compute tax	Items + tax rate	Returns subtotal × rate
81	Invoice	calculateTotal()	Compute total	Items + tax	Returns subtotal + tax
82	Invoice	setDate()	Update date	Valid date	Date updated
83	Invoice	equals()	Compare invoices	Same id	true
84	Order	setStatus()	Update order status	Valid new status S	Status updated
85	Order	setStatus()	Invalid status	Null or unknown	Throws exception
86	Order	assignEmployee()	Assign employee	Valid employee	Employee assigned
87	Order	addItem()	Add order item	Valid	Item added
88	Order	getTotalItems()	Count items	Multiple items	Correct count
89	Order	calculateTotal()	Sum items	Valid order	Total computed correctly

90	Order	scheduleDeliver y()	Set date	Valid date	Delivery date set
91	Order	scheduleDeliver y()	Invalid date (past)	Date < today	Throws exception
92	Order	equals()	CompaCompare ordersre orders	Same id	Returns true
93	Order	equals()	CompaCompare ordersre orders	Different id	Returns false
94	Order	cancel()	Cancel order	Status = CANCELE D	Items locked, status updated
95	Order	reopen()	Reopen canceled order	CANCELE D → OPEN	Status updated
96	Order	reopen()	Invalid reopen	Delivered order	Throws exception
97	Inventory	auditStock()	Verify stock	Called on item	Returns stock snapshot
98	Inventory	setPrice()	Change price	Positive number	Price updated
99	Inventory	setPrice()	Change price	Negative	Throws exception
10 0	Employee	toString()	Represent as string	Valid employee	Formatted string returned